

Improving Tree of Thoughts with Batching and Refinement

Minghan Gao (mg2328), Warren Hua (wsh48), Zhichun Zhang (zz547), Yihan Zhao (yz2788)

Github: https://github.com/Y1hhnn/CS4782_Team78

1. Introduction

Large language models are often used through direct input-output (IO) prompting or chain-of-thought (CoT) prompting [2], but these approaches generate solutions along a single left-to-right path. One bad early step can ruin the final answer, with no mechanism to recover. Our project reproduces and extends “*Tree of Thoughts: Deliberate Problem Solving with Large Language Models*” by Yao et al. [1], which addresses this limitation. ToT turns reasoning into a search over intermediate “thoughts”: it generates multiple candidates, evaluates partial states via LM self-assessment, and uses BFS or DFS to explore, prune, and backtrack. However, the paper does not discuss how evaluation structure (batched vs. unbatched prompting [3, 4]) affects self-assessment, nor whether vote-based selection is optimal for open-ended tasks. We reproduce all three tasks using modern Gemini models and extend the analysis along two axes: (1) how batching the evaluator degrades accuracy and what remedial strategies recover it, and (2) whether score-based candidate selection outperforms vote-based planning.

2. Chosen Results

We targeted the original paper’s main quantitative results across all three tasks: the paper’s Table 2 for Game of 24 (ToT $b = 5$ reaches 74% on GPT-4, far above IO 7.3% and CoT 4%), Figures 4-5 for Creative Writing (testing ToT with no exact verifier), and Table 3 for Mini Crosswords (testing ToT-DFS with up to 10 steps requiring backtracking). These collectively test ToT under different search algorithms (BFS vs. DFS), thought granularities (equations vs. words vs. plans), and evaluation modes (value classification vs. voting).

3. Methodology

Game of 24. We re-implemented ToT’s three-module pipeline (generator, evaluator, BFS controller) in Python, calling Gemini 3.1 Flash-Lite (thinking) and Gemini 2.5 Flash-Lite (non-thinking) via Vertex AI [8]. All prompts are verbatim from the original code [6]. Our verifier uses SymPy [9] for arithmetic and operand-multiset checking. A key challenge was discovering that the paper’s evaluation strategy (one API call per candidate) is never stated in the text but only implicit in the code. Our initial batched implementation (~20 calls/puzzle) silently degraded accuracy, motivating our central extension: a systematic comparison of *batched* vs. *unbatched* (~170 calls/puzzle) evaluation, a batch-size sweep ($bs \in \{1, 2, 4, 8, 16, 40\}$, $n = 20$), verdict distribution analysis over 500 candidates, and a zero-cost verifier tiebreaker that uses local symbolic checking to resolve ties among the evaluator’s top-scored candidates.

Mini Crosswords. We re-implemented ToT’s DFS-based crossword solver using Gemini 3.1. Each thought fills a single word clue. The solver translates filled words into letter constraints and prompts the LM five times for candidate words with confidence levels, aggregated into a priority queue. For state evaluation, each remaining clue is assessed as sure/maybe/impossible, and if any is deemed impossible, the subtree is pruned and DFS backtracks. We used the same 20 puzzles (indices 1, 6, ..., 96) from GooBix[7], averaging up to 5 runs, measuring letter, word, and game accuracy. We ran the full ablation suite (-prune, -backtrack, +best state) and extended with three call shapes (unbatched/half/full batch) and four evaluator modes (basic, focused with optional top-K, verified, and a per-puzzle value cache).

Creative Writing. We evaluated 100 text inputs, each with four required ending sentences, using Gemini 2.5 Flash-Lite as the generator and Gemini 3.1 Flash-Lite Preview as a fixed LLM-as-judge (1-10 coherence score). Beyond reproducing IO, CoT, and ToT baselines, we added a post-generation refinement step and introduced a selection-mechanism ablation (plan-only CoT, plan-vote, standard ToT, ToT score-select) to isolate the effects of planning, voting, and independent scoring. We also ran best-of-k controls (best-of-k IO, self-consistency CoT) to test whether gains come from tree-structured reasoning or simply from sampling multiple candidates and selecting the best.

4. Results & Analysis

Game of 24. Unbatched ToT ($b = 5$) exceeds the paper’s GPT-4 accuracy: 87% on Gemini 2.5 and 96% on Gemini 3.1, vs. 74% in the original (Figure 1). However, batching the value evaluator collapses Gemini 2.5 from

87% to 15% while Gemini 3.1 drops only from 96% to 88%. Verdict analysis over 500 depth-0 candidates (Figure 2) reveals the mechanism: batched Gemini 2.5 votes “impossible” on 66% of candidates (vs. 0% unbatched), shifting from absolute to competitive grading, while batched Gemini 3.1 maintains 62% “sure” verdicts, suggesting its thinking tokens provide robustness against list-position bias. Unbatching costs 3-4× more (Table 1). Our zero-cost verifier tiebreaker (local symbolic checking) fully recovers Gemini 3.1 (88% → 96%) but only partially recovers Gemini 2.5 (15% → 28%), diagnosing that batching corrupts only the final selection on thinking models but the entire BFS trajectory on non-thinking models.

Mini Crosswords. Gemini 3.1 closely matches GPT-4: IO 45.4% letter / 18.3% word (vs. 38.7% / 14.0%), CoT 38.8% / 15.0% (vs. 40.6% / 15.6%), unbatched ToT 76.6% / 59.1% / 35.0% game vs. paper 78.0% / 60.0% / 20.0% — nearly 2× game (Table 2). Ablations: −backtrack collapses to 5.0% game in 3.6 steps (vs. 23.7), −prune drops to 17.5%, but +best-state adds zero (35.0% → 35.0%) — unlike the paper, Gemini’s DFS commits cleanly to the gold grid rather than walking past it. For the batching extension (Figure 5), full batching cuts calls by 65% for an 11pp game drop (35.0% → 24.0%); half batching is dominated (25.0% game) despite the highest letter accuracy (77.6%). Focused candidate-level verdicts (Figure 4) strictly dominate basic at both levels: full focused 27.5% / 118 calls, half focused k7 36.2% / 421 calls. An unbatched value cache wins through regularization and becomes the accuracy leader: 40.0% game / 347 calls / −23% runtime (Table 3, Table 4). Verified regressed every focused configuration, as it un-kills focused’s high-precision pruning decisions.

Creative Writing. Standard ToT achieved the highest baseline score (mean = 6.94), outperforming IO (6.63) and CoT (6.50), though all showed high variance (Figure 6). Compared with the original paper (ToT = 7.56, CoT = 6.93, IO = 6.19), our results preserve the ranking but with more moderate separation, possibly because Gemini’s stronger baseline generation leaves less room for search to help. Refinement slightly improved all methods (ToT+Refine: 7.14, CoT+Refine: 6.82). The selection-mechanism ablation (Figure 7) reveals that ToT score-select achieved the highest coherence (8.14) with the most concentrated distribution, substantially outperforming vote-based ToT (6.94). Critically, best-of-k IO (7.93) and best-of-k CoT (7.61) also exceeded standard ToT (Figure 8), suggesting the primary gain comes not from tree structure or voting but from generating multiple candidates and selecting the best via independent scoring.

5. Reflections

Game of 24. The most important lesson is that faithful reproduction requires replicating engineering context the paper never highlights. The per-state evaluation pattern is implicit in the code, never discussed in the text. Our failure to reproduce led directly to the batching catastrophe as our central finding, transforming a debugging exercise into a study of how LLM-as-judge behavior shifts under multi-candidate prompting. What began as “why don’t our numbers match?” became a characterization of absolute vs. competitive evaluation modes, with implications for any system using LLMs to score candidates.

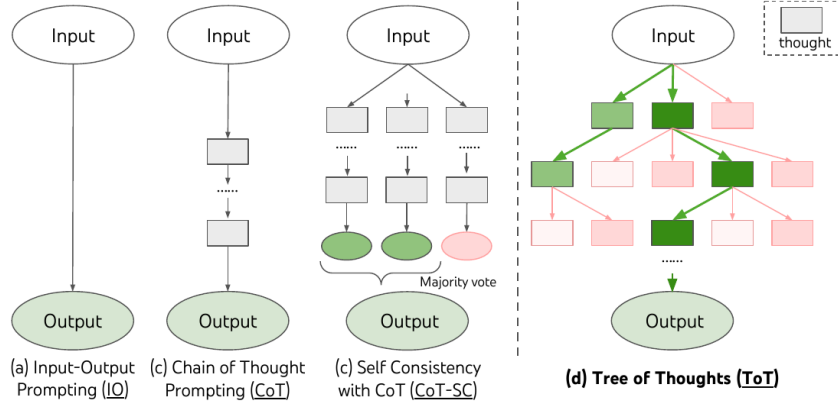
Mini Crosswords. Crosswords confirmed backtracking as ToT’s primary engine; without it the solver collapses to greedy filling. The zero +best-state gap is the more interesting finding: unlike the paper, Gemini’s DFS commits cleanly to the gold grid rather than walking past it. Focused candidate-level verdicts Pareto-dominate per-clue scoring at both batch levels, and an unbatched value cache was the cheapest accuracy win in the study by regularizing noisy gold-prompt verdicts across siblings; verified, conversely, backfired.

Creative Writing. Creative Writing gains are not driven only by ToT’s tree structure. The best-of-k and score-select results indicate that candidate sampling and reliable selection matter more than vote-based planning, likely because the lack of a deterministic verifier makes evaluator quality the bottleneck. Standard ToT commits early to one voted plan, limiting diversity. Score-select preserves and independently evaluates multiple outputs. This connects to human-AI writing interfaces like CoAuthor [5], where surfacing multiple suggestions lets the writer compare and adapt.

Cross-task synthesis. Across all three tasks, ToT’s value decomposes into (1) the search structure (BFS/DFS with back-tracking) and (2) the evaluation mechanism. For constrained tasks (Game of 24, Crosswords), search dominates, and backtracking and pruning are essential. For open-ended tasks (Creative Writing), evaluation dominates, and independent scoring outperforms voting. Future ToT implementations should match the evaluation strategy to task type. With more time, we would combine focused verdicts with the cache for crosswords and fine-tune a small evaluator to reduce Game of 24 costs. Total cost: ~\$15.

Figures and Tables

Reproduced Original Results (Yao et al. [1])



Original Figure 1. (From [1].) Schematic illustrating IO, CoT, CoT-SC, and ToT approaches to problem solving with LLMs. ToT maintains a search tree over intermediate thoughts.

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	74%
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

Original Table 2. (From [1].) Game of 24 results with GPT-4. ToT (b = 5) achieves 74% success, far above IO (7.3%) and CoT (4.0%).

Method	Success Rate (%)		
	Letter	Word	Game
IO	38.7	14	0
CoT	40.6	15.6	1
ToT (ours)	78	60	20
+best state	82.4	67.5	35
-prune	65.4	41.5	5
-backtrack	54.6	20	5

Table 3: Mini Crosswords results.

Original Table 3. (From [1].) Mini Crosswords results with GPT-4. ToT achieves 78% letter, 60% word, and 20% game accuracy.

Our Results

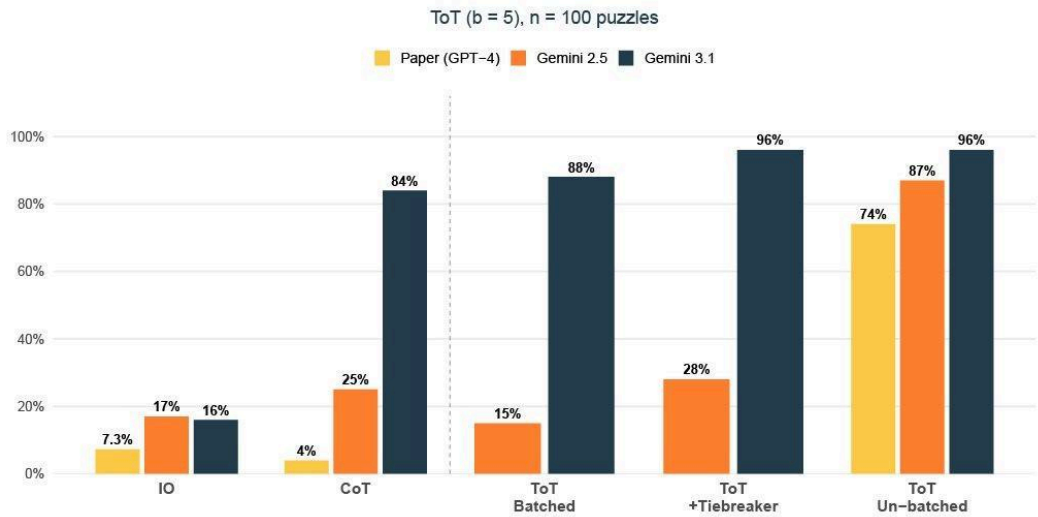


Figure 1. Success rate on Game of 24 (n = 100, b = 5). Unbatched ToT on both Gemini models exceeds GPT-4’s 74%. Batching collapses Gemini 2.5; the tiebreaker partially recovers it at zero cost.

Cost of Un-batching (ToT b = 5)

Gem 3.1 thinking tokens make un-batching 4.6× more expensive

	Cost / 100 puzzles	Time / puzzle	API calls / puzzle
Gem 2.5 Batched	\$0.32	~8 s	~20
Gem 2.5 Un-batched	\$0.99	~70 s	~170
Gem 3.1 Batched	\$0.77	~12 s	~20
Gem 3.1 Un-batched	\$3.52	~90 s	~170

Table 1. Cost of unbatching for ToT (b = 5). Unbatching costs 3–4× more. Gemini 3.1 thinking tokens make unbatching 4.6× more expensive.

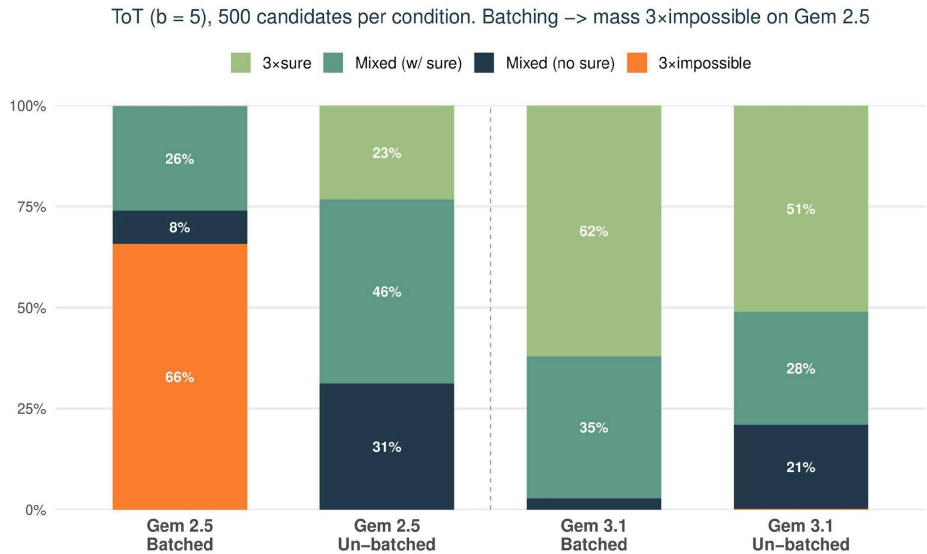


Figure 2. Evaluator verdict distribution at BFS depth 0. Batching causes Gemini 2.5 to vote “impossible” on 66% of candidates (vs. 0% unbatched). Gemini 3.1’s thinking resists: 62% “sure” even when batched.

Avg. Success Rate (%) of 5 Runs of 20 Puzzle Games	Letter Acc.		Word Acc.		Game Acc.		Avg. Steps (Ours)	Avg. API Calls (Ours)
	Paper (GPT-4)	Ours (Gemini 3.1)	Paper	Ours	Paper	Ours		
IO	38.7%	45.9%	14.0%	18.5%	0.0%	0.0%	1.0	1.0
CoT	40.6%	38.8%	15.6%	15.2%	1.0%	1.0%	1.0	1.0
ToT (unbatched)	78.0%	77.4%	60.0%	59.1%	20.0%	33.0%	24.3	461.9
→ + best state	82.4%	81.2%	67.5%	64.0%	35.0%	37.1%	19.9	225.68
→ - prune	65.4%	69.13%	41.5%	46.5%	5.0%	20.0%	18.6	103.25
→ - backtrack	54.6%	37.5%	20.0%	22.2%	5.0%	3.3%	3.5	26.0

Table 2. Mini Crosswords results: average success rate (%) over 5 runs of 20 puzzle games. Gemini 3.1 matches or exceeds GPT-4 on all metrics.

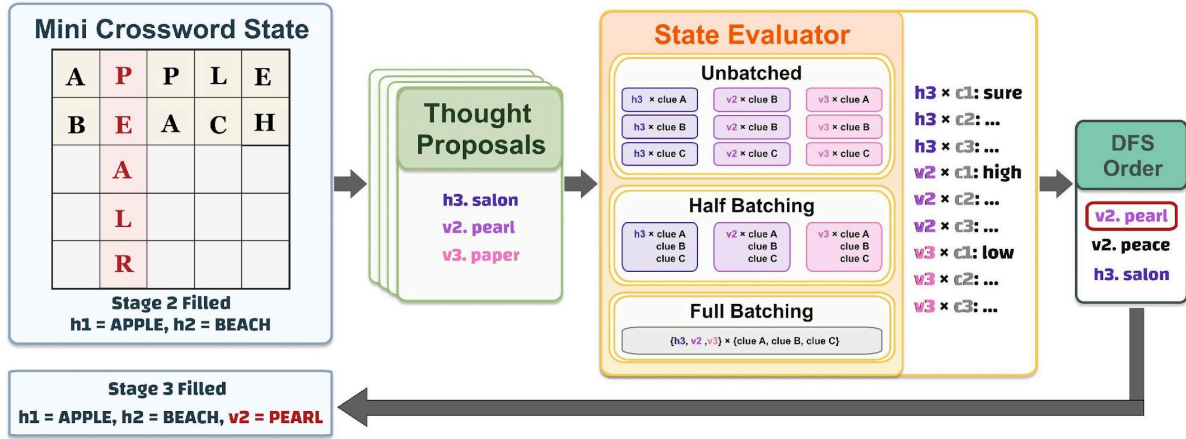


Figure 3. Crosswords batching schematic. Thought proposals are evaluated under three call shapes: unbatched, half batch, and full batch—with decreasing cost but increasing list-position noise.

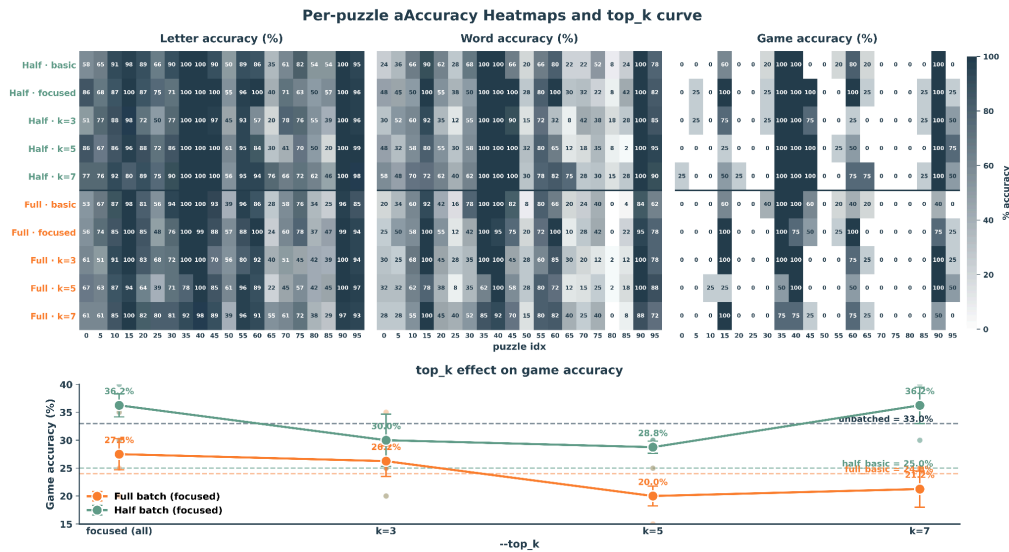


Figure 4. Focused evaluator: per-puzzle heatmaps (top) and top-K sweep on game accuracy (bottom). Focused Pareto-dominates basic on both batch levels; half_focused_k7 matches unbatched, full peaks at top-K=0.

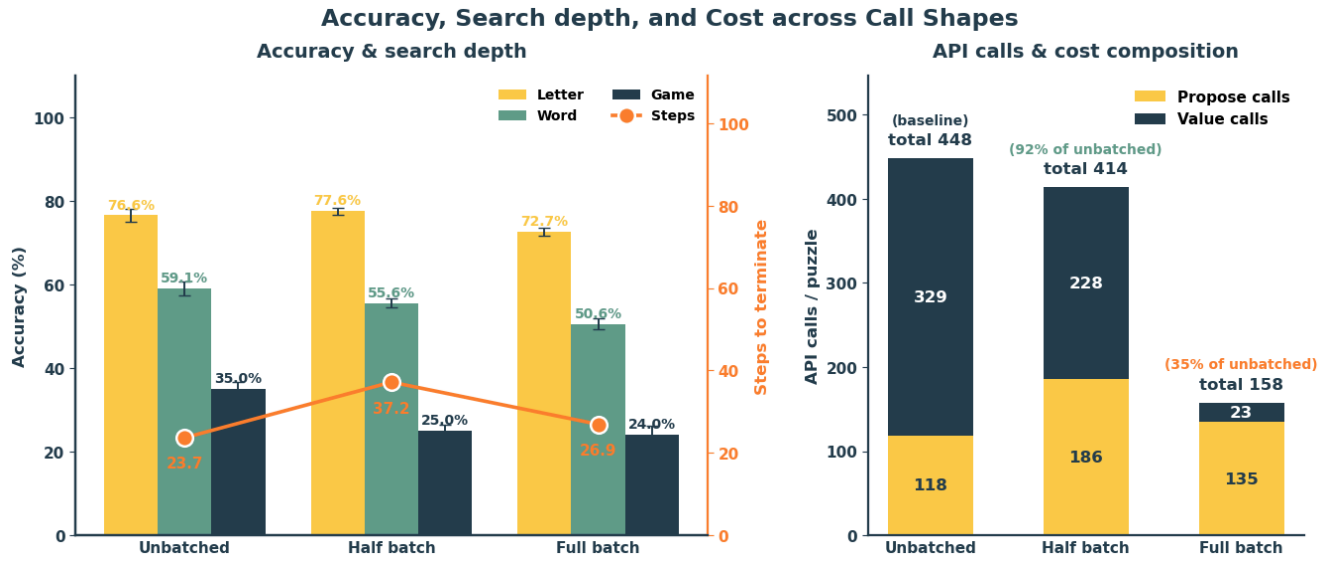


Figure 5. Accuracy, search depth, and API cost across call shapes. Full batching cuts cost by 65% but drops game accuracy from 35.0% to 24.0%. Half batching achieves the highest letter accuracy (77.6%).

Config	Game $\pm$ SE	Letter	Word	Calls	Propose	Value	Steps
Unbatched (paper)	33.0 $\pm$ 2.5	77.4 $\pm$ 1.6	59.1 $\pm$ 1.5	461.9 $\pm$ 48.1	121.5 $\pm$ 6.5	340.4 $\pm$ 44.3	24.3 $\pm$ 1.3
Unbatched + cache	40.0 $\pm$ 4.1	77.1 $\pm$ 0.8	61.5 $\pm$ 2.5	347.2 $\pm$ 39.0	135.2 $\pm$ 18.2	212.0 $\pm$ 21.0	27.0 $\pm$ 3.6

Table 3: Unbatched (paper) vs. unbatched + value cache. The cache cuts value calls 38% and runtime 23%, and the +7pp game gain (40.0% vs. 33.0%) lies within the SE band.

Tier	Config	Game %	Calls	Cost vs paper-unbatched
Cost-bound	full_focused (k=0)	27.5	117.7	24%
Mid	half_focused (k=0)	36.2	404.8	81%
Mid	half_focused_k7	36.2	420.6	84%
New	unbatched_cache	40.0	347.2	69%
Paper-strict	unbatched	33.0	461.9	92%

Table 4. Cost-vs-accuracy tier summary across the crossword study. Unbatched + cache Pareto-dominates the paper-strict configuration on both axes.

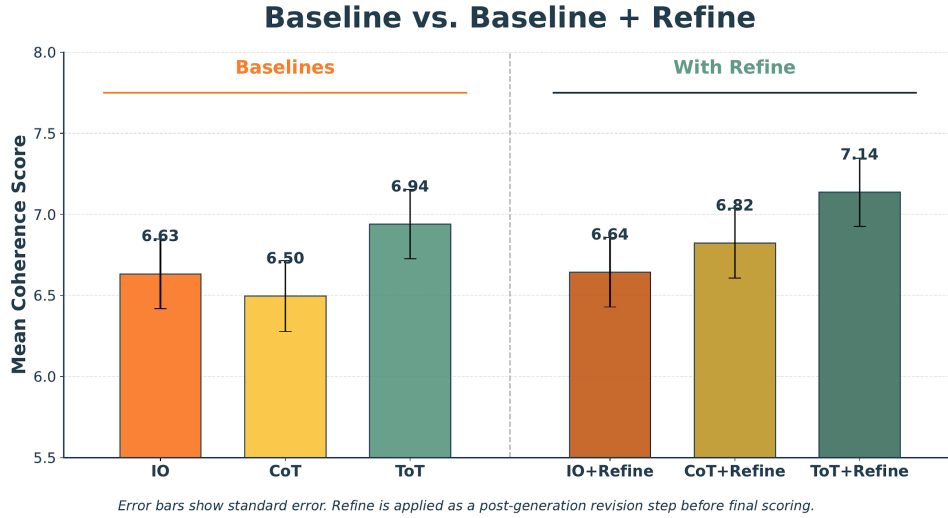


Figure 6. Baseline vs. Baseline + Refine on Creative Writing. Refinement modestly improves all methods; ToT+Refine achieves the highest score (7.14).

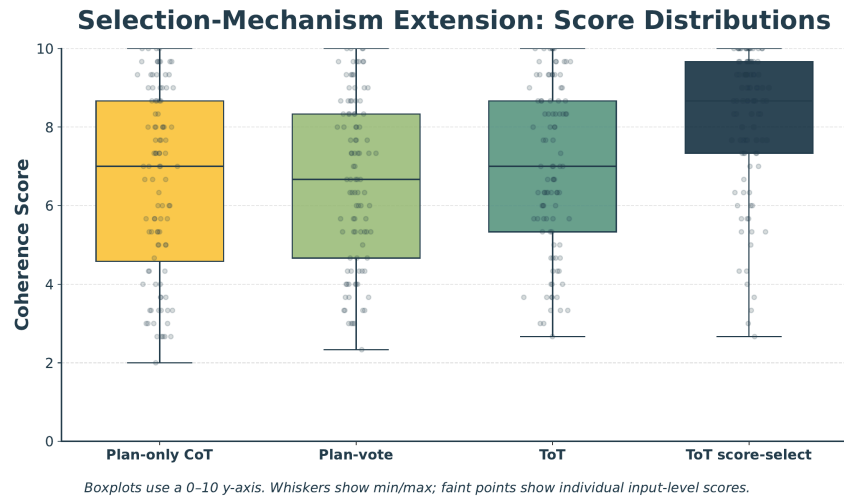


Figure 7. Selection-mechanism ablations: score distributions across plan-only CoT, plan-vote, standard ToT, and ToT score-select. Score-select concentrates scores near the top of the range.

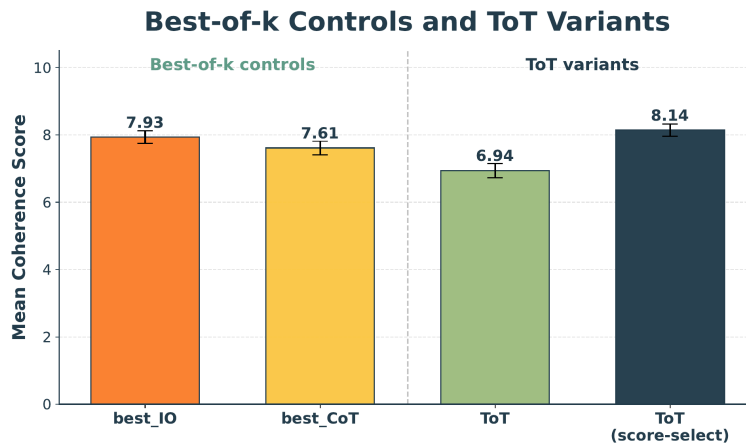


Figure 8. Best-of-k controls vs. ToT variants. Best-of-k IO (7.93) and ToT score-select (8.14) both outperform standard vote-based ToT (6.94).

References

- [1] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” NeurIPS, 2023.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.
- [3] Z. Cheng et al., “Batch Prompting: Efficient Inference with LLM APIs,” EMNLP Industry Track, 2023.
- [4] J. Lin et al., “BatchPrompt: Accomplish More with Less,” ICLR, 2024.
- [5] M. Lee, P. Liang, & Q. Yang, “CoAuthor: Designing a Human-AI Collaborative Writing Dataset,” CHI, 2022.
- [6] ToT reference code: github.com/princeton-nlp/tree-of-thought-llm
- [7] Puzzle data: 4nums.com (Game of 24), GooBix (Crosswords)
- [8] Google Cloud Vertex AI (Gemini 2.5 Flash-Lite, Gemini 3.1 Flash-Lite)
- [9] SymPy: symbolic mathematics library for Python